

Activity 5 - Organise a crate into modules

AI Vibe Coding with Rust / TGS-2023039924 / version 3.0 / K2/A2

Specification

money::format_cents(u64) -> String renders "\$D.CC". report::report_line(name, qty, unit_cents) -> Option<String> renders "<name> x<qty> @ <unit> = <total>"; report::report joins lines with "\n" and fails wholly on overflow.

Plan before code

Act as a senior Rust reviewer. Specification: money::format_cents(u64) -> String renders "\$D.CC". report::report_line(name, qty, unit_cents) -> Option<String> renders "<name> x<qty> @ <unit> = <total>"; report::report joins lines with "\n" and fails wholly on overflow. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Organise a crate into modules" in starter/src so that the supplied tests pass. Keep the two-module layout and public paths; do not merge modules to make tests pass. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.